

COLLABORATIVE
BUCKET LIST APPLICATION
AND TRANSFER PROTOCOL

PROJECT INTRODUCTION

Project: Collaborative Bucket List Application

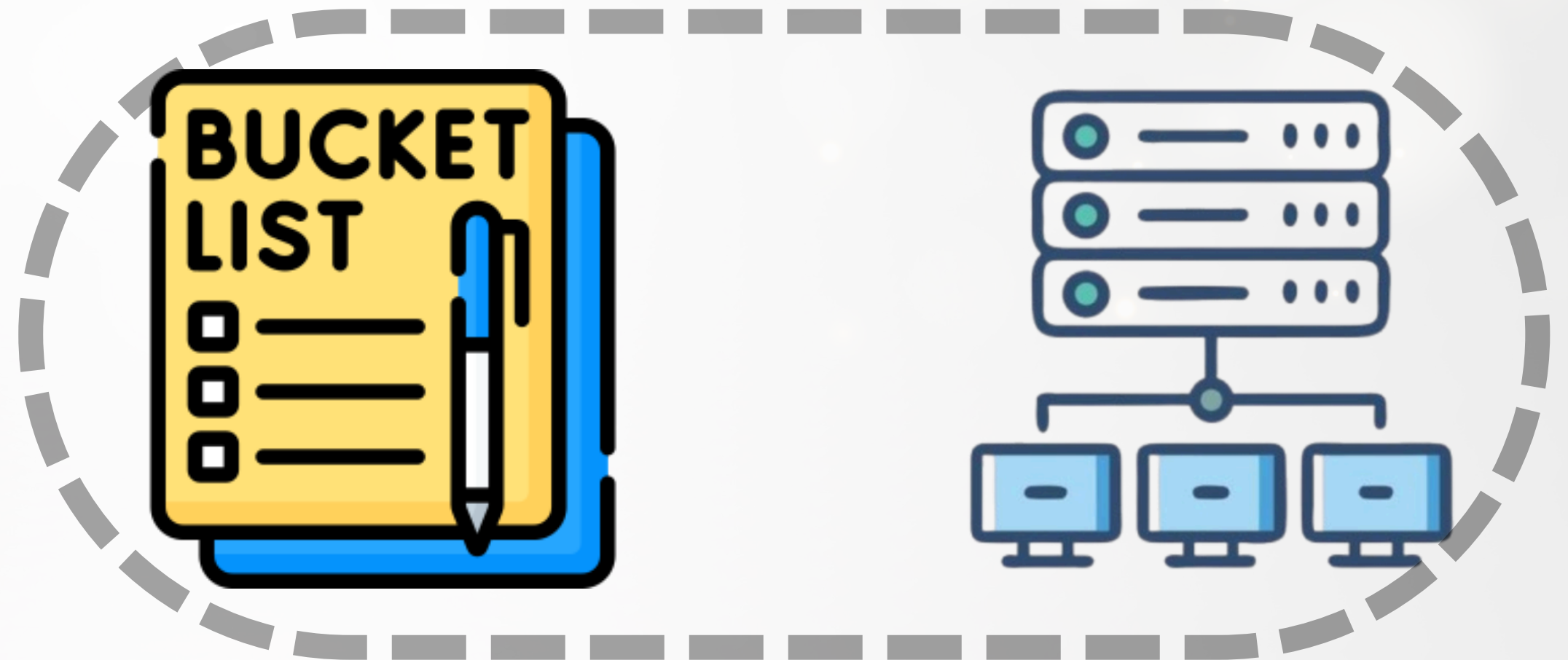
Architecture: Client-Server

Transport Layer: TCP

Application-Layer Protocol: CBLP

Main Concept:

A network application that allows multiple users to connect to the same server and collaboratively manage a shared Bucket List.



PROJECT OBJECTIVE

The objective of this project is to develop a network application that allows users to manage a shared Bucket List through a network which allows the user to.

- Create an account system with a username and password
- Properly authenticate before using the application
- Add/Remove/Edit shared Bucket List items and description
- View the shared Bucket List among users
- See other users currently connected to the system
- Allow multiple users to use such functions in shared bucket list at the same time while also notifying every other clients of the user's activity

APPLICATION STRUCTURE

1

CLIENT

- CONNECTS TO SERVER USING TCP
- ACCEPTS USER COMMANDS
- CREATES CBLP REQUESTS
- DISPLAY RESPONSE AND NOTIFICATIONS

2

SERVER

- ACCEPTS TCP CONNECTIONS
- HANDLES MULTIPLE CLIENTS
- PROCESSES CBLP REQUESTS
- MAINTAINS ACCOUNTS AND BUCKET LIST
- SENDS RESPONSES AND NOTIFICATIONS

3

PROTOCOL

- DEFINES CBLP MESSAGE STRUCTURE
- DEFINES STATUS CODES AND PHRASES
- ENCODES/DECODES JSON MESSAGES

APPLICATION CHARACTERISTIC

CLIENT-SERVER ARCHITECTURE

A central Server manages users and the shared Bucket List.

MUTIPLE- CLIENTS SUPPORT

Multiple users can connect to the Server simultaneously.

SHARED APPLICATION STATE

The Bucket List is maintained by the Server and shared between clients.

USER AUTHENTICATION SYSTEM

Users must register and authenticate before using the Bucket List.

REQUEST-RESPONSE COMMUNICATION

Clients send requests and the Server returns responses. The Server can also notify other connected clients when the shared list changes.

SERVER AND CLIENT ROLES IN APPLICATION

SERVER

- Managing accounts
- Authenticating users
- Maintaining the Bucket List
- Processing requests
- Sending responses
- Broadcasting changes to connected clients

CLIENT

- Accepting commands list from the server
- Sending requests to the server
- Receiving and displaying responses
- Displaying notifications

TRANSPORT LAYER: TCP

This project uses TCP (Transmission Control Protocol) as the main Transport Layer service

Why TCP?

The application contains various information that needs to be delivered correctly such as.

- Username
- Password
- Bucket List items
- Descriptions
- Add/Edit/Remove operations

Losing or corrupting this information could cause the shared application state to become **incorrect**.

Therefore, **reliable communication** is **much** more important than minimizing latency.

TCP ADVANTAGES OVER UDP

1

RELIABLE DATA TRANSFER
TCP PROVIDES RELIABLE DELIVERY OF DATA.

2

ORDERED DELIVERY
MESSAGES ARRIVE AT APPLICATION IN THE SAME ORDER.

3

ERROR HANDLING
TCP HANDLES PACKET LOSS AND RETRANSMISSION AUTOMATICALLY.

APPLICATION LAYER PROTOCOL: CBLP

Collaborative Bucket List Protocol

CBLP is this project's Application-Layer Protocol designed specifically for the Collaborative Bucket List Application.

CBLP Defines

- Request types
- Request format
- Response format
- Status codes
- Status phrase
- Payload structure
- Server notification

CBLP Operates on top of TCP

Collaborative Bucket List

CBLP (Application-Layer)

TCP (Transport Layer)

IP

CBLP MESSAGE FORMATS

Collaborative Bucket List Protocol uses “JSON” as it’s message format.

Request (Client > Server)

```
{
  "request": "ADD",
  "payload": {
    "title": "Visit Japan",
    "description": "Travel around Tokyo and Kyoto"
  }
}
```

Response (Server > Client)

```
{
  "status": 201,
  "phrase": "CREATED",
  "request": "ADD",
  "payload": {}
}
```

```
[201 CREATED] ADD
{
  "item": {
    "id": 1,
    "title": "Visit Japan",
    "description": "Travel around Tokyo and Kyoto",
    "added_by": "Alice"
  }
}
```




CBLP AVAILABLE REQUESTS

REGISTER

CREATE AN ACCOUNT

AUTH

AUTHENTICATE A USER

ADD

ADD BUCKET LIST ITEM

LIST

DISPLAY ALL ITEMS

VIEW

DISPLAY ONE ITEM

EDIT

EDIT AN ITEM

REMOVE

REMOVE AN ITEM

CLEAR

CLEAR THE LIST

USERS

DISPLAY CONNECTED USERS

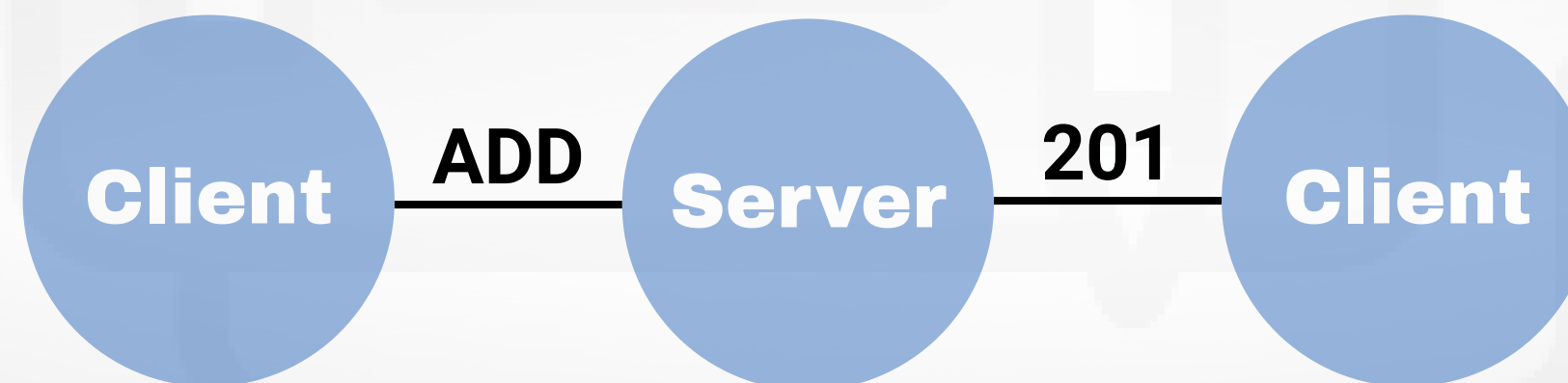
QUIT

END CONNECTION



RESPONSE STATUS CODES

200	OK	REQUEST SUCCESSFUL
201	CREATED	NEW RESOURCE CREATED
400	BAD_REQUEST	INVALID REQUEST/DATA
401	UNAUTHORIZED	AUTHENTICATION FAILED/REQUIRED
404	NOT_FOUND	RESOURCE DOES NOT EXIST
409	CONFLICT	REQUEST CONFLICT WITH STATE
500	INTERNAL_SERVER_ERROR	UNEXPECTED SERVER ERROR





USAGE DEMONSTRATION

```
MINGW64:/c/Users/Jakkarin

Jakkarin@LAPTOP-E5N7SD85 MINGW64 ~
$ python '/c/Users/Jakkarin/Desktop/Network Project/server.py'
Bucket List Server running on 0.0.0.0:8888
[CONNECT] 127.0.0.1:63861
[CONNECT] 127.0.0.1:63862
```

```
MINGW64:/c/Users/Jakkarin

Jakkarin@LAPTOP-E5N7SD85 MINGW64 ~
$ python '/c/Users/Jakkarin/Desktop/Network Project/client.py'
Collaborative Bucket List
Commands:
/register USERNAME PASSWORD
/auth USERNAME PASSWORD
/add "ITEM" "OPTIONAL DESCRIPTION"
/add "ITEM"
/list
/view ID
/edit ID "NEW TITLE" "NEW DESCRIPTION"
/remove ID
/clear
/users
/quit
> |
```

```
MINGW64:/c/Users/Jakkarin

Jakkarin@LAPTOP-E5N7SD85 MINGW64 ~
$ python '/c/Users/Jakkarin/Desktop/Network Project/client.py'
Collaborative Bucket List
Commands:
/register USERNAME PASSWORD
/auth USERNAME PASSWORD
/add "ITEM" "OPTIONAL DESCRIPTION"
/add "ITEM"
/list
/view ID
/edit ID "NEW TITLE" "NEW DESCRIPTION"
/remove ID
/clear
/users
/quit
>
```

```
MINGW64:/c/Users/Jakkarin

Jakkarin@LAPTOP-E5N7SD85 MINGW64 ~
$ python '/c/Users/Jakkarin/Desktop/Network Project/server.py'
Bucket List Server running on 0.0.0.0:8888
[CONNECT] 127.0.0.1:63861
[CONNECT] 127.0.0.1:63862
```

```
MINGW64:/c/Users/Jakkarin

/register Furry fb123
Invalid command or arguments
> /register Furry fb123
>
[201 CREATED] REGISTER
{
  "message": "Account created",
  "username": "Furry"
}
> /auth Furry fb123
>
[200 OK] AUTH
{
  "message": "Authentication successful",
  "username": "Furry"
}
> |
```

```
MINGW64:/c/Users/Jakkarin

Jakkarin@LAPTOP-E5N7SD85 MINGW64 ~
$ python '/c/Users/Jakkarin/Desktop/Network Project/client.py'
Collaborative Bucket List
Commands:
/register USERNAME PASSWORD
/auth USERNAME PASSWORD
/add "ITEM" "OPTIONAL DESCRIPTION"
/add "ITEM"
/list
/view ID
/edit ID "NEW TITLE" "NEW DESCRIPTION"
/remove ID
/clear
/users
/quit
> auth BabyB 123
Invalid command or arguments
> /auth BabyB 123
>
[404 NOT_FOUND] AUTH
{
  "message": "Account not found"
}
```

```
MINGW64:/c/Users/Jakkarin

/quit
> auth BabyB 123
Invalid command or arguments
> /auth BabyB 123
>
[404 NOT_FOUND] AUTH
{
  "message": "Account not found"
}
> auth Furry fb123
Invalid command or arguments
> /auth furry fb123
>
[404 NOT_FOUND] AUTH
{
  "message": "Account not found"
}
> /auth Furry fb123
>
[409 CONFLICT] AUTH
{
  "message": "Account is already logged in"
}
> |
```

```
MINGW64:/c/Users/Jakkarin

/users
/quit
> register Furry fb123
Invalid command or arguments
> /register Furry fb123
>
[201 CREATED] REGISTER
{
  "message": "Account created",
  "username": "Furry"
}
> /auth Furry fb123
>
[200 OK] AUTH
{
  "message": "Authentication successful",
  "username": "Furry"
}
>
[200 OK] USER_JOINED
{
  "username": "goybeammaster"
}
> |
```




USAGE DEMONSTRATION

ADD REQUEST

```
MINGW64:/c/Users/Jakkarin
> /auth Furry fb123
[200 OK] AUTH
{
  "message": "Authentication successful",
  "username": "Furry"
}
[200 OK] USER_JOINED
{
  "username": "goybeammaster"
}
> /add "Create Werewolf Panting" "Preferably handsome-looking one"
[201 CREATED] ADD
{
  "item": {
    "id": 1,
    "title": "Create Werewolf Panting",
    "description": "Preferably handsome-looking one",
    "added_by": "Furry"
  }
}
[201 CREATED] REGISTER
{
  "message": "Account created",
  "username": "goybeammaster"
}
> /auth goybeammaster palantir
[200 OK] AUTH
{
  "message": "Authentication successful",
  "username": "goybeammaster"
}
[200 OK] ITEM_ADDED
{
  "item": {
    "id": 1,
    "title": "Create Werewolf Panting",
    "description": "Preferably handsome-looking one",
    "added_by": "Furry"
  }
}
```

EDIT REQUEST

```
MINGW64:/c/Users/Jakkarin
"username": "goybeammaster"
> /add "Create Werewolf Panting" "Preferably handsome-looking one"
[201 CREATED] ADD
{
  "item": {
    "id": 1,
    "title": "Create Werewolf Panting",
    "description": "Preferably handsome-looking one",
    "added_by": "Furry"
  }
}
[200 OK] ITEM_UPDATED
{
  "item": {
    "id": 1,
    "title": "Make smoked Meat",
    "description": "Preferably sausages",
    "added_by": "Furry"
  }
}
[200 OK] ITEM_ADDED
{
  "item": {
    "id": 1,
    "title": "Create Werewolf Panting",
    "description": "Preferably handsome-looking one",
    "added_by": "Furry"
  }
}
> /edit 1 "Make smoked Meat"
Invalid command or arguments
> /edit 1 "Make smoked Meat" "Preferably sausages"
[200 OK] EDIT
{
  "message": "Item updated",
  "item": {
    "id": 1,
    "title": "Make smoked Meat",
    "description": "Preferably sausages",
    "added_by": "Furry"
  }
}
```

LIST,QUIT REQUEST

```
MINGW64:/c/Users/Jakkarin
{
  "message": "Wrong password"
}
> /auth Furry fb123
[200 OK] AUTH
{
  "message": "Authentication successful",
  "username": "Furry"
}
> list
Invalid command or arguments
> /list
[200 OK] LIST
{
  "items": []
}
[200 OK] USER_LEFT
{
  "username": "goybeammaster"
}
[200 OK] LIST_CLEARED
{
  "by": "Furry"
}
[200 OK] USER_LEFT
{
  "username": "Furry"
}
[200 OK] USER_JOINED
{
  "username": "Furry"
}
> quit
Invalid command or arguments
> /quit
Jakkarin@LAPTOP-E5N7SD85 MINGW64 ~
$
```


A top-down view of a desk with various items: a laptop in the upper center, a cup of coffee in the upper left, a pen in the center, a pair of glasses in the lower center, several paper clips on the left, and a large green leaf in the bottom left corner. The background is a light, neutral color.

THANK YOU